

Bayesian Deep Learning for Continuous Adaptation and Anomaly Detection in Radio Frequency Signals: An Exhaustive Review (2020–2025)

Executive Summary

The modern radio frequency (RF) spectrum is a hyper-dynamic environment characterized by non-stationary spectral activity, rapid proliferation of heterogeneous wireless devices, and increasingly sophisticated adversarial threats. Traditional signal processing pipelines and static Deep Neural Networks (DNNs) are proving inadequate for this domain because they operate on the flawed assumption of distributional stationarity between training and deployment phases. This report presents a comprehensive, expert-level analysis of the integration of Bayesian probability theory with Deep Learning (BDL) to engineer "conscious" RF systems capable of quantifying their own uncertainty, detecting concept drift in real-time, and triggering autonomous continuous learning loops.

Focusing on the critical window of research between 2020 and 2025, we synthesize advancements in Uncertainty Quantification (UQ), Bayesian Online Change Point Detection (BOCPD), and hybrid neural architectures. We specifically examine the application of these methods to raw I/Q (In-phase and Quadrature) data for tasks such as Automatic Modulation Classification (AMC), RF fingerprinting, and wireless intrusion detection. The analysis reveals that distinguishing between *aleatoric uncertainty* (inherent channel noise) and *epistemic uncertainty* (model ignorance due to novel signals) is the linchpin for effective anomaly detection.

We detail the architectural evolution from computationally prohibitive full Bayesian Neural Networks (BNNs) to efficient hybrid models utilizing Bayesian Last Layers (BLLs) and Deep Ensembles. Furthermore, we deconstruct the algorithmic mechanisms—specifically Uncertainty-Regularized Continual Learning (UCL) and feature reconstruction metrics—that utilize high epistemic uncertainty scores as gating signals to initiate retraining, thereby solving the stability-plasticity dilemma in edge-deployed RF sensors. This report serves as a definitive technical guide for implementing self-adaptive RF machine learning systems using modern probabilistic programming libraries like Pyro, GPyTorch, and TensorFlow Probability.

1. Introduction: The Stochastic Nature of the Spectral Domain

The intersection of Deep Learning and Signal Processing has yielded remarkable improvements in the classification and analysis of radio frequency signals. Deep Neural Networks (DNNs), particularly Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), have demonstrated the ability to learn hierarchical feature representations directly from raw complex-valued I/Q samples, bypassing the need for rigid, handcrafted statistical features like higher-order cumulants or cyclic moments.¹ However, the operational reality of wireless communications—spanning 5G, IoT, and tactical networks—presents a fundamental challenge to the standard supervised learning paradigm: the assumption of Independent and Identically Distributed (i.i.d.) data.

In the RF domain, the data distribution is inherently non-stationary. Channel conditions fluctuate due to multipath fading, Doppler shifts, and atmospheric variability. Hardware impairments in transmitter circuitry drift due to thermal aging. Moreover, the spectral landscape is populated by uncooperative emitters and potential jammers that introduce signal types never seen during training. When a standard deterministic DNN encounters such Out-of-Distribution (OOD) inputs, it fails catastrophically. Crucially, it fails silently and overconfidently. Softmax output probabilities, often misinterpreted as confidence, tend to saturate even for inputs far removed from the training manifold, leading to high-confidence errors in modulation classification or device authentication.³

To address this effectively, we must move beyond point estimates of model parameters and predictions. Bayesian Deep Learning (BDL) provides a principled mathematical framework for this transition. By treating network weights as random variables described by probability distributions, BDL enables the quantification of predictive uncertainty. This report posits that uncertainty is not merely a metric for model reliability but a functional control signal. It is the necessary feedback mechanism to detect *concept drift*—the point where the underlying data generation process shifts—and to trigger *continuous learning*, allowing the model to adapt to the new spectral reality without forgetting its prior knowledge.

We structure this review to first establish the mathematical rigor of uncertainty quantification methods applied to time-series data. We then explore how these methods are operationalized for drift detection through hybrid architectures that decouple feature extraction from probabilistic reasoning. Finally, we analyze specific implementations in the RF domain, detailing the loss functions, metrics, and software ecosystems that enable the deployment of these adaptive systems.

2. Mathematical Foundations of Uncertainty in Time-Series

Understanding the utility of Bayesian methods for anomaly detection requires a rigorous distinction between the sources of uncertainty and the specific approximations used to compute them in high-dimensional parameter spaces.

2.1 The Dichotomy of Uncertainty: Aleatoric vs. Epistemic

In the context of time-series anomaly detection for RF, the total predictive uncertainty is the sum of two distinct components, each requiring different handling strategies.

Aleatoric Uncertainty captures the irreducible noise inherent in the data generation process. In RF signals, this corresponds to thermal noise, interference, and the stochastic nature of the wireless channel (e.g., Rayleigh or Rician fading). It is a property of the data, not the model. Even with infinite training data, aleatoric uncertainty remains. Mathematically, for an input x and output y , this is modeled as a distribution $p(y|x)$ (e.g., the variance σ^2 in a regression task or the entropy of the categorical distribution in classification).³

Epistemic Uncertainty, or model uncertainty, arises from a lack of knowledge. It corresponds to the uncertainty in the model parameters θ due to limited data. In spectral monitoring, high epistemic uncertainty indicates that the received signal falls into a region of the I/Q feature space that is sparsely populated in the training set—signaling a potential anomaly, a new modulation scheme, or an adversarial attack. Unlike aleatoric uncertainty, epistemic uncertainty can be reduced by gathering more data in that region.³

The decomposition of uncertainty is critical for triggers. A system should not trigger retraining (continuous learning) on high aleatoric uncertainty (which would just fit noise), but it *must* trigger on high epistemic uncertainty (which indicates a gap in knowledge).

2.2 Variational Inference (VI) and the Evidence Lower Bound

Full Bayesian inference involves computing the posterior distribution of the weights $p(\theta|\mathcal{D})$ given the dataset $\mathcal{D}$:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})}$$

The marginal likelihood $p(\mathcal{D})$ involves an integral over all possible parameter configurations, which is intractable for modern DNNs with millions of parameters. Variational Inference (VI) transforms this integration problem into an optimization problem. We posit a family of approximate distributions $q_\phi(\theta)$ (variational distribution), parameterized by ϕ , and minimize the Kullback-Leibler (KL) divergence between $q_\phi(\theta)$ and the true posterior $p(\theta|\mathcal{D})$.⁸

Minimizing $\text{KL}(q_\phi(\theta) \parallel$

$p(\theta|\mathcal{D}))$ is mathematically equivalent to maximizing the Evidence Lower Bound (ELBO), a pervasive loss function in the literature reviewed:

$$\mathcal{L}_{ELBO}(\phi) = \mathbb{E}[q_\phi(\theta)] - \text{KL}(q_\phi(\theta) \parallel$$

$|p(\theta)|$

- **First Term (Likelihood Cost):** Encourages the model to fit the data (minimize reconstruction error or cross-entropy).
- **Second Term (Complexity Cost):** Regularizes the model by forcing the approximate posterior to stay close to the prior $p(\theta)$, often a standard Gaussian.

In time-series anomaly detection, VI is often applied to Recurrent Neural Networks (Bayesian LSTMs). Weights in the LSTM gates are sampled from $q_\phi(\theta)$ during each forward pass. The variance of the resulting predictions over multiple samples provides the epistemic uncertainty estimate. However, the computational cost of sampling weights for every layer during training and inference has led researchers to seek more efficient approximations.¹⁰

2.3 Monte Carlo (MC) Dropout: The Practical Bayesian Approximation

MC Dropout remains a dominant technique in the 2020–2025 literature due to its simplicity and effectiveness. It reinterprets the standard dropout regularization technique as an approximate Bayesian inference in a deep Gaussian Process. By keeping dropout active during the inference phase, the network becomes stochastic.³

For an input I/Q sequence x^* , we perform T stochastic forward passes. The predictive mean and epistemic uncertainty (variance) are estimated as:

$$\mathbb{E}[y^*] \approx \frac{1}{T} \sum_{t=1}^T \hat{y}_t^*$$

$$\text{Var}(y^*) \approx \frac{1}{T} \sum_{t=1}^T (\hat{y}_t^* - \mathbb{E}[y^*])^2 + \tau^{-1}$$

where τ relates to the precision of the observation noise. In modulation classification, this manifests as running the classifier multiple times on the same signal burst. If the classification flips between QPSK and 8-PSK across runs, the epistemic uncertainty is high, flagging the signal as ambiguous or anomalous.¹²

2.4 Deep Ensembles

Deep Ensembles involve training M independent deterministic networks with different random initializations and potentially on different shuffles of the data. The predictive distribution is a mixture of the individual network outputs.

$$p(y|x) = \frac{1}{M} \sum_{m=1}^M p_{\theta_m}(y|x)$$

Recent empirical studies, including those focused on implicit ensembles in large neural networks, indicate that Deep Ensembles often provide better calibrated uncertainty estimates than MC Dropout or even VI, particularly for OOD detection.³ They tend to capture multimodality in the posterior better than the unimodal Gaussian approximations often used in

VI. In the context of RF, an ensemble of CNNs trained on signal modulation data has been shown to produce predictive distributions where the width (variance) correlates strongly with the error rate, effectively capturing the uncertainty induced by low SNR or adversarial perturbations.¹ The primary drawback is the linear increase in computational and memory cost with the number of ensemble members (\$M\$), which poses a challenge for edge deployment on Software Defined Radios (SDRs).

2.5 Evidential Deep Learning (EDL)

To circumvent the computational overhead of sampling (MC Dropout) or training multiple models (Ensembles), Evidential Deep Learning (EDL) has gained traction. EDL places a distribution over the class probabilities themselves—specifically, a Dirichlet distribution for classification tasks. The network predicts the hyperparameters α of this Dirichlet distribution directly.⁷

The uncertainty is derived deterministically from the "evidence" collected by the network.

$$u = \frac{K}{\sum_{k=1}^K \alpha_k}$$

where K is the number of classes. EDL allows for a single forward pass to yield both a prediction and a measure of epistemic uncertainty. Recent applications in high-energy physics and anomaly detection suggest EDL is computationally efficient but requires careful calibration of the evidence regularizer in the loss function to prevent the model from generating high evidence for incorrect classes in OOD scenarios.⁷

Table 1: Comparison of Uncertainty Quantification Methods for RF Time-Series

Method	Uncertainty Type	Computational Cost (Inference)	Memory Cost	Suitability for RF Edge	Primary Loss Function
MC Dropout	Epistemic	High (T passes)	Low (1 model)	Medium	Standard Cross-Entropy/MSE
Deep Ensembles	Aleatoric + Epistemic	High (M passes)	High (M models)	Low (unless distilled)	Negative Log-Likelihood (NLL)
Variational	Epistemic	Medium/High	Medium (2x params for	Medium	ELBO (Evidence

Inference		(Sampling)	μ, σ		Lower Bound)
Evidential DL	Epistemic	Low (1 pass)	Low (1 model)	High	EDL Loss (MSE + KL divergence)
Bayesian Last Layer	Epistemic	Low (Analytic)	Low	High	Marginal Likelihood / ELBO

3. Drift Detection: From Statistical Tests to Neural-Bayesian Hybrids

Detecting when the RF environment has shifted (concept drift) is the precursor to adaptation. While traditional statistical process control charts (e.g., CUSUM) detect shifts in mean or variance of univariate streams, RF data is high-dimensional. Modern approaches leverage deep networks to reduce dimensionality before applying probabilistic drift detection.

3.1 Bayesian Online Change Point Detection (BOCPD)

BOCPD is a recursive message-passing algorithm that infers the posterior distribution of the "run length" r_t , defined as the time elapsed since the last change point. At each time step t , it calculates the probability that the current run length is zero (a change point occurred) given the observed data $x_{1:t}$.¹⁴

$$p(r_t | x_{1:t}) \propto p(r_t | r_{t-1}) p(x_t | r_{t-1}, x_{1:t-1}^{(r)})$$

- **Hazard Function:** $p(r_t | r_{t-1})$ defines the prior probability of a change point occurring at any given step (often modeled as a constant hazard rate or a geometric distribution).
- **Predictive Probability:** $p(x_t | r_{t-1}, x_{1:t-1}^{(r)})$ is the likelihood of the current data point given the data associated with the current run.

In raw signal processing, applying BOCPD directly to I/Q samples is computationally intractable and statistically weak due to noise. The state-of-the-art solution, as identified in recent literature, is **Neural-BOCPD** or **Probabilistic Predictive Coding (PPC)**.¹⁵ Here, a DNN (e.g., an RNN or Transformer) acts as a feature extractor, compressing the sequence into a latent representation z_t . BOCPD is then applied to the stream of latent vectors or to the sequence of predictive losses (surprisal).

3.2 Hybrid Architectures: The DNN + Autoencoder Drift Detector

A prominent architecture for detecting concept drift in streams is the **DNN+AE-DD** (Deep Neural Network + Autoencoder Drift Detection). This hybrid approach addresses the high-dimensional nature of the data by using an Autoencoder to learn the manifold of "normal" behavior.¹⁶

Architecture:

1. **Base Classifier (DNN):** Trained on the initial distribution to perform the primary task (e.g., modulation classification).
2. **Drift Detector (Autoencoder):** An identical or auxiliary network trained to reconstruct the hidden layer representations of the base classifier.

Mechanism:

The system monitors the Reconstruction Error ($L_{\text{recon}} = \|z - \hat{z}\|^2$). Under normal conditions, the latent features z lie on the manifold the Autoencoder has learned, resulting in low reconstruction error. When concept drift occurs (e.g., a new signal type appears), the base classifier generates latent features that deviate from the learned manifold. The Autoencoder fails to reconstruct these novel features accurately, causing a spike in reconstruction error.

Metric for Drift:

The sequence of reconstruction errors is analyzed using a statistical threshold, such as the Drift Detection Method (DDM) or the 3-sigma rule:

$$\mu_{\text{error}} \pm 3\sigma_{\text{error}}$$

If the error rate exceeds the warning level, the system enters a "warning" state, buffering data. If it breaches the drift level, a concept drift is confirmed, and the retraining trigger is activated.¹⁶ This method decouples the task of prediction from the task of monitoring, utilizing the "projection" of data into the latent space as a robust metric for drift.

3.3 Probabilistic Scores for High-Dimensional Streams

Beyond reconstruction error, papers from 2020-2025 utilize sophisticated probabilistic scores to quantify drift:

- **Wasserstein Distance:** In scenarios with "co-drifting" features, simple correlation checks fail. The Wasserstein distance (or Earth Mover's Distance) measures the work required to transform the distribution of incoming windowed data into the reference training distribution. It is particularly robust to noise in RF signals compared to KL divergence, providing a smoother gradient for detecting gradual drift.¹¹
- **Feature Reconstruction Error (FRE):** Utilized in Continual Open-world Uncertainty Quantification (COUQ), FRE measures the L_2 norm difference between a feature vector and its reconstruction via Principal Component Analysis (PCA) bases learned for

each known class. High FRE indicates the sample belongs to a "novel" class, effectively distinguishing epistemic uncertainty (novelty) from aleatoric noise.¹⁸

- **Maximum Mean Discrepancy (MMD):** A kernel-based statistical test used to determine if two samples are drawn from different distributions. It is often employed in the validation phase of drift detection to confirm that a buffered set of "anomalous" samples indeed constitutes a statistically significant new distribution requiring model adaptation.¹⁵

4. Continuous Learning Triggers: The Feedback Loop

The "conscious" RF system must not only detect anomalies but act upon them. This section synthesizes the mechanisms by which uncertainty scores are transformed into triggers for the retraining loop, creating a closed-loop continuous learning system.

4.1 The Epistemic Trigger Mechanism

Traditional triggers based on validation loss are reactive and require ground truth labels, which are unavailable in real-time RF streams. Uncertainty-based triggers are proactive and unsupervised.

1. The Uncertainty Threshold Trigger:

This mechanism monitors the rolling average of the epistemic uncertainty metric (e.g., Mutual Information from a Deep Ensemble or Entropy from a BNN).

$$\bar{U}_t = \alpha U_t + (1-\alpha) \bar{U}_{t-1}$$

A trigger event is generated when $\bar{U}_t > \tau_{\text{retrain}}$.

- **Logic:** A sustained increase in epistemic uncertainty implies the model is consistently encountering data it does not understand. This distinguishes genuine drift from transient noise spikes (aleatoric uncertainty), which would be smoothed out by the rolling average.¹

2. The BOCPD Confidence Trigger:

Systems utilizing Bayesian Online Change Point Detection provide a probability of run length $P(r_t=0)$. The trigger is activated only when this probability exceeds a high confidence threshold (e.g., 0.95).

- **Logic:** This prevents "thrashing" (frequent, unnecessary retraining) by ensuring that a structural change in the data generation process has statistically occurred before investing computational resources in retraining.¹⁴

3. The Buffer-Fill Trigger (Active Learning):

In hybrid systems, samples flagged with high epistemic uncertainty are not immediately discarded. They are stored in a "Replay Buffer" or "Novelty Buffer." The retraining loop is triggered when this buffer reaches a critical capacity N_{buffer} .

- **Logic:** This ensures the retraining batch has sufficient statistical significance to update

the model weights effectively without overfitting to a few outliers.²⁰

4.2 Uncertainty-Regularized Continual Learning (UCL)

Once triggered, the retraining process faces the "Catastrophic Forgetting" problem—learning the new jammer profile might erase the ability to classify friendly signals.

Uncertainty-regularized Continual Learning (UCL) addresses this by using the Bayesian posterior variance of the weights themselves as a regularization term.²²

Mechanism:

For each weight w_i in the BNN, we have a mean μ_i and a standard deviation σ_i (uncertainty).

- **High Uncertainty (σ_i is large):** The weight is not critical for previous tasks. It is "plastic" and allowed to change significantly during retraining.
- **Low Uncertainty (σ_i is small):** The weight is critical for previous knowledge. It is "stable" and heavily regularized to prevent change.

The loss function for UCL incorporates a Kullback-Leibler divergence term that penalizes deviations from the "prior" (which is the posterior of the previous task), weighted by the inverse of the uncertainty:

$$\mathcal{L}_{UCL} \approx \sum_i \frac{(w_i - \mu_{i, old})^2}{2\sigma_{i, old}^2}$$

This allows the RF model to surgically update only the parts of its network relevant to the new drift while preserving its core capabilities—a crucial requirement for long-term deployment in wireless security.²²

5. Architectural Innovations for RF Anomaly Detection

This section applies the theoretical concepts to specific RF architectures found in the 2020-2025 literature, identifying the exact loss functions and metrics used.

5.1 Deep Ensembles for Automatic Modulation Classification (AMC)

Context: Identifying modulation types (BPSK, QPSK, 16QAM) in low SNR environments where signals are easily confused.

Paper: An Uncertainty Quantification Framework for Deep Learning-Based Automatic Modulation Classification.¹

Architecture: A Deep Ensemble of Convolutional Neural Networks (CNNs). Each member processes raw I/Q sequences.

Loss Function: Negative Log-Likelihood (NLL). The ensemble minimizes the NLL of the predictive distribution, effectively maximizing the likelihood of the true class while calibrating the output probabilities.

$$\mathcal{L} = -\sum_{n=1}^N \log p(y_n | x_n, \theta)$$

Uncertainty Metric: Mutual Information (MI). The system calculates the MI between the prediction and the model parameters. High MI indicates that the ensemble members disagree significantly (high model uncertainty), effectively flagging OOD signals (e.g., a jammer or unknown modulation) even if the average Softmax confidence is moderate.

Key Insight: The ensemble approach produces "wider" confidence intervals for incorrectly predicted signals compared to standard BNNs, offering a superior safety margin for decision-making in cognitive radio systems.

5.2 DeepCRF: Channel-Resilient RF Fingerprinting

Context: Authenticating devices based on unique hardware impairments (fingerprints) embedded in the Channel State Information (CSI) or I/Q data. Concept drift occurs when the wireless channel changes (e.g., moving from Line-of-Sight to Non-Line-of-Sight).

Paper: DeepCRF: Deep Learning-Enhanced CSI-Based RF Fingerprinting.²⁴

Architecture: A CNN backbone utilizing Supervised Contrastive Learning.

Loss Function: Supervised Contrastive Loss.

$$\mathcal{L}_{con} = \sum_{i \in I} \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(z_i \cdot z_p / \tau)}{\sum_{a \in A(i)} \exp(z_i \cdot z_a / \tau)}$$

This loss forces latent representations of the same device (across different channel conditions) to cluster tightly together while pushing different devices apart.

Drift/Metric: While primarily a representation learning technique, the distance in the embedding space (e.g., Euclidean or Cosine distance to class centroids) serves as the drift metric. An incoming signal that maps far from all known device clusters is flagged as an anomaly (potential spoofer) or a drift in the device's own fingerprint due to aging. This triggers a retraining or enrollment process.

5.3 Hybrid Bayesian Last Layer (BLL) Models

Context: General RF time-series prediction and anomaly detection where full BNNs are too slow.

Paper: Improved uncertainty quantification for neural networks with Bayesian last layer.²⁷

Architecture: A deterministic Deep Neural Network (e.g., ResNet or LSTM) acting as a feature extractor, feeding into a Bayesian Linear Regression layer at the output.

Loss Function: Marginal Likelihood Maximization (Type-II Maximum Likelihood).

The system optimizes the deterministic weights to maximize the marginal likelihood of the data under the Bayesian interpretation of the last layer.

Uncertainty Metric: Predictive Variance (Analytic).

$$\sigma^2_{\text{pred}} = \sigma^2_{\text{noise}} + \phi(x)^T \Sigma_{\text{post}} \phi(x)$$

Here, $\phi(x)$ is the feature vector from the deterministic network. The second term is the epistemic uncertainty. If the feature vector $\phi(x)$ projects into a direction not spanned by the training data, the quadratic form $\phi^T \Sigma \phi$ explodes, instantly flagging the anomaly.

Key Insight: This architecture represents the "sweet spot" for RF engineering—offering the feature learning of Deep Learning with the rigorous, calibrated uncertainty of Gaussian Processes, but with inference speeds compatible with real-time SDR constraints.

5.4 Efficient RF Passive Component Modeling

Context: Modeling complex RF components (filters) where obtaining training data (EM simulations) is expensive.

Paper: Efficient RF Passive Components Modeling with Bayesian Online Learning.²⁹

Architecture: Bayesian Neural Network with reconfigurable heads.

Trigger Mechanism: Uncertainty-Aware Adaptive Sampling.

The system uses the BNN's uncertainty map over the design space to trigger new EM simulations. Instead of a retraining loop for drift, this is an active learning loop: new data is generated only where the model is uncertain.

Metric: Epistemic uncertainty variance of the BNN output.

6. Implementation Landscape: Libraries and Frameworks

Implementing these sophisticated Bayesian architectures requires specialized probabilistic programming libraries that integrate seamlessly with deep learning frameworks.

6.1 Pyro (Built on PyTorch)

Pyro is the industry standard for academic BDL research. It enables the definition of generative models using simple Python functions.

- **Role in RF:** Used to implement full BNNs and Variational Inference. The `pyro.infer.SVI` module automates the optimization of the ELBO loss function.
- **Key Feature:** `pyro.nn.PyroModule` allows users to "Bayesianize" a standard PyTorch `nn.Module` by replacing deterministic weights with `pyro.sample` statements (priors).
- **Drift Detection:** Pyro's logic allows for complex probabilistic definitions of change points, making it suitable for implementing custom Neural-BOCPD models.³⁰

6.2 GPyTorch

GPyTorch is highly optimized for Gaussian Processes (GPs) and Deep Kernel Learning.

- **Role in RF:** Essential for implementing **Bayesian Last Layer** architectures and Deep Gaussian Processes. It uses Blackbox Matrix-Matrix (BBMM) multiplication to scale GP

inference to large datasets, which is critical for processing high-rate I/Q streams.

- **Key Feature:** Seamless integration of Deep Neural Networks as feature extractors (Kernels) for GPs, enabling the hybrid architectures discussed in Section 3.2.³²

6.3 TyXe

TyXe is a lightweight library built on top of Pyro specifically designed for Bayesian Neural Networks.

- **Role in RF:** It simplifies the code required to turn a standard PyTorch network (e.g., a ResNet for AMC) into a BNN. It abstracts the complexity of defining priors and guides, allowing RF engineers to focus on architecture rather than probabilistic boilerplate. It supports local reparameterization, which reduces gradient variance and speeds up training.³³

6.4 Relevant GitHub Repositories

- **DeepCRF Implementation:** https://github.com/Oriseven/DeepCRF_TIFS - Contains PyTorch code for the DeepCRF framework, including contrastive loss implementations and data augmentation pipelines for CSI data.³⁴
- **Uncertainty-Regularized Continual Learning (UCL):** <https://github.com/csm9493/UCL> - Provides the official TensorFlow implementation of the UCL algorithm, demonstrating how to compute node-wise uncertainty and apply the regularized loss function.²²
- **Deep Learning for Anomaly Detection:** <https://github.com/bitzhangcy/Deep-Learning-Based-Anomaly-Detection> - A comprehensive repository of anomaly detection architectures, including Autoencoder-based drift detectors suitable for time-series adaptation.³⁶

7. Critical Analysis and Future Directions

While the integration of Bayesian methods into RF deep learning has matured significantly between 2020 and 2025, several critical challenges remain for widespread deployment.

7.1 The Computational Bottleneck at the Edge

Most BDL methods (Deep Ensembles, MC Dropout) require multiple forward passes or expensive sampling operations. RF systems often operate on edge devices (FPGAs, embedded SDRs) with strict power and latency budgets.

- **Future Direction:** Research into **Quantized Bayesian Neural Networks** and **Distillation** techniques is crucial. Distilling the knowledge (and uncertainty) of a Deep Ensemble into a single deterministic student network could retain the benefits of UQ while meeting the microsecond latency requirements of 6G physical layer control.

7.2 The "Uncertainty Attack" Vector

As RF systems become reliant on uncertainty triggers, they open a new attack surface. An adversary could craft "Uncertainty Attacks"—signals designed specifically to maximize the model's epistemic uncertainty without being classified as a threat, effectively jamming the *learning process* rather than the receiver.

- **Future Direction:** Developing **Adversarially Robust BNNs** that can distinguish between natural epistemic uncertainty and adversarially induced ambiguity is a frontier for wireless security research.

7.3 Calibration in Non-Stationary Regimes

Most uncertainty calibration techniques (e.g., Temperature Scaling) assume a static validation set. In a continuously drifting RF environment, a model calibrated at time t may be miscalibrated at time $t + \Delta$.

- **Future Direction: Online Calibration** methods that dynamically adjust the uncertainty output based on the stream's characteristics are needed to ensure the retraining triggers remain reliable over long operational periods.

8. Conclusion

The convergence of Bayesian probability and Deep Learning represents a pivotal evolution in the design of cognitive Radio Frequency systems. The static "train-and-deploy" paradigm is obsolete in the face of the modern, chaotic spectral environment. The literature from 2020–2025 establishes **epistemic uncertainty quantification** as the enabling technology for **Continuous Learning**.

By rigorously distinguishing between the noise of the channel and the novelty of the signal, Bayesian models provide the necessary high-fidelity signals to trigger automated retraining loops. Hybrid architectures—specifically the **DNN + Bayesian Last Layer** and **Neural-BOCPD**—emerge as the optimal design patterns, balancing the feature-extraction power of Deep Learning with the analytical tractability of Bayesian inference. Supported by mature libraries like Pyro and GPyTorch, and driven by algorithms like UCL and DeepCRF, these "conscious" radio systems are poised to define the intelligence of next-generation wireless networks, capable of autonomous adaptation and resilient anomaly detection in the most challenging spectral conditions.

9. Appendix: Technical Summary of Key Algorithms

9.1 Uncertainty-Regularized Continual Learning (UCL) Algorithm

Based on ²²

Objective: Update model parameters θ on new task t without forgetting tasks $1 \dots t-1$.

Initialization: Pre-trained Bayesian Network with means μ and variances σ^2 (uncertainty).

Loss Function:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{task}}(\theta) + \lambda \sum_{l=1}^L \sum_i \frac{(\mu_{t,i}^{(l)} - \mu_{t-1,i}^{(l)})^2}{\sigma_{t-1,i}^{(l)2}}$$

Process:

1. **Drift Detected:** Trigger retraining on new data batch.
2. **Regularization:** Weights with small σ_{t-1}^2 (certain weights) incur a high penalty for changing (Stability). Weights with large σ_{t-1}^2 (uncertain weights) incur low penalty (Plasticity).
3. **Update:** Optimize μ_t, σ_t via backpropagation.
4. **Result:** Model adapts to drift while preserving "certain" prior knowledge.

9.2 DeepCRF Drift Detection Logic

Based on ²⁴

Objective: Detect if incoming signal x is from a known device D_{known} or a new/rogue device D_{rogue} despite channel changes.

Model: Encoder $E(\cdot)$ trained with Supervised Contrastive Loss.

Process:

1. **Inference:** Compute embedding $z = E(x)$.
2. Distance Calculation: Compute distance d to nearest class centroid c_k in embedding space.
$$d = \min_k \|z - c_k\|_2$$
3. **Uncertainty Score:** Convert distance to probability (e.g., via OpenMax or simply $1/(1+d)$).
4. **Thresholding:**
 - If $d < \tau_{\text{match}}$: Classified as Device k .
 - If $d > \tau_{\text{novelty}}$: Flag as **Epistemic Anomaly** (New Device/Drift).
5. **Trigger:** If anomaly count $>$ threshold, trigger enrollment/retraining for new class.

Works cited

1. An Uncertainty Quantification Framework for Deep Learning-Based Automatic Modulation Classification - arXiv, accessed January 18, 2026, <https://arxiv.org/html/2503.04142v2>
2. Deep learning-based automated modulation classification for cognitive radio | Request PDF, accessed January 18, 2026, https://www.researchgate.net/publication/312963470_Deep_learning-based_auto

- [mated_modulation_classification_for_cognitive_radio](#)
3. Epistemic Uncertainty Collapse in Large Models - arXiv, accessed January 18, 2026, <https://arxiv.org/pdf/2409.02628>
 4. Evaluation of Uncertainty Quantification in Deep Learning - PMC, accessed January 18, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC7274324/>
 5. From Aleatoric to Epistemic: Exploring Uncertainty Quantification Techniques in Artificial Intelligence - arXiv, accessed January 18, 2026, <https://arxiv.org/html/2501.03282>
 6. A Review of Bayesian Uncertainty Quantification in Deep Probabilistic Image Segmentation, accessed January 18, 2026, <https://arxiv.org/html/2411.16370v4>
 7. Evidential Deep Learning for Uncertainty Quantification and Out-of-Distribution Detection in Jet Identification using Deep Neural Networks - arXiv, accessed January 18, 2026, <https://arxiv.org/html/2501.05656v1>
 8. Implicit versus explicit Bayesian priors for epistemic uncertainty estimation in clinical decision support - PMC - NIH, accessed January 18, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12306758/>
 9. STUDENT-T PROCESSES AS INFINITE-WIDTH LIMITS OF POSTERIOR BAYESIAN NEURAL NETWORKS - OpenReview, accessed January 18, 2026, <https://openreview.net/pdf?id=iUppQcPAMK>
 10. Comparison of Uncertainty Quantification with Deep Learning in Time Series Regression - the University of Groningen research portal, accessed January 18, 2026, https://research.rug.nl/files/558372600/paper_uncertainty_time_series.pdf
 11. Adaptive Concept Drift Detection Using Bayesian Neural Networks and Wasserstein Distance | Request PDF - ResearchGate, accessed January 18, 2026, https://www.researchgate.net/publication/393924891_Adaptive_Concept_Drift_Detection_Using_Bayesian_Neural_Networks_and_Wasserstein_Distance
 12. Uncertainty-Aware Incremental Automatic Modulation Classification With Bayesian Neural Network | Request PDF - ResearchGate, accessed January 18, 2026, https://www.researchgate.net/publication/379875927_Uncertainty-Aware_Incremental_Automatic_Modulation_Classification_with_Bayesian_Neural_Network
 13. An Uncertainty Quantification Framework for Deep Learning-Based Automatic Modulation Classification - ResearchGate, accessed January 18, 2026, https://www.researchgate.net/publication/389647710_An_Uncertainty_Quantification_Framework_for_Deep_Learning-Based_Automatic_Modulation_Classification
 14. [0710.3742] Bayesian Online Change-point Detection - arXiv, accessed January 18, 2026, <https://arxiv.org/abs/0710.3742>
 15. Anomalous Change Point Detection Using Probabilistic Predictive Coding - arXiv, accessed January 18, 2026, <https://arxiv.org/html/2405.15727v2>
 16. Concept Drift Detection Based on Deep Neural Networks and Autoencoders - MDPI, accessed January 18, 2026, <https://www.mdpi.com/2076-3417/15/6/3056>
 17. Rajeev Sahay's research works | University of California San Diego and other places, accessed January 18, 2026, <https://www.researchgate.net/scientific-contributions/Rajeev-Sahay-2150644313>
 18. Uncertainty Quantification in Continual Open-World Learning, accessed January

- 18, 2026, <https://arxiv.org/abs/2412.16409>
19. Efficacy of Statistical and Artificial Intelligence-based False Information Cyberattack Detection Models for Connected Vehicles - arXiv, accessed January 18, 2026, <https://arxiv.org/pdf/2108.01124>
 20. [1703.02910] Deep Bayesian Active Learning with Image Data - arXiv, accessed January 18, 2026, <https://arxiv.org/abs/1703.02910>
 21. Federated Learning Under Concept Drift: A Systematic Survey of Foundations, Innovations, and Future Research Directions - MDPI, accessed January 18, 2026, <https://www.mdpi.com/2079-9292/14/22/4480>
 22. Uncertainty-based Continual Learning with Adaptive Regularization - NIPS, accessed January 18, 2026, <http://papers.neurips.cc/paper/8690-uncertainty-based-continual-learning-with-adaptive-regularization.pdf>
 23. Uncertainty-based Continual Learning with Adaptive Regularization - arXiv, accessed January 18, 2026, <https://arxiv.org/pdf/1905.11614>
 24. DeepCRF: Deep Learning-Enhanced CSI-Based RF Fingerprinting for Channel-Resilient WiFi Device Identification - arXiv, accessed January 18, 2026, <https://arxiv.org/html/2411.06925v1>
 25. DeepCRF: Deep Learning-Enhanced CSI-Based RF Fingerprinting for Channel-Resilient WiFi Device Identification - arXiv, accessed January 18, 2026, <https://arxiv.org/pdf/2411.06925>
 26. Towards Channel-Resilient CSI-Based RF Fingerprinting using Deep Learning - arXiv, accessed January 18, 2026, <https://arxiv.org/pdf/2403.15739>
 27. Improved Uncertainty Quantification for Neural Networks With ..., accessed January 18, 2026, <https://d-nb.info/1326662090/34>
 28. Reliably Detecting Model Failures in Deployment Without Labels - arXiv, accessed January 18, 2026, <https://arxiv.org/html/2506.05047v4>
 29. [2511.15125] Efficient RF Passive Components Modeling with Bayesian Online Learning and Uncertainty Aware Sampling - arXiv, accessed January 18, 2026, <https://arxiv.org/abs/2511.15125>
 30. Deep probabilistic programming with Pyro - Broad Institute, accessed January 18, 2026, <https://www.broadinstitute.org/talks/deep-probabilistic-programming-pyro>
 31. Making Your Neural Network Say "I Don't Know" — Bayesian NNs using Pyro and PyTorch, accessed January 18, 2026, <https://medium.com/data-science/making-your-neural-network-say-i-dont-know-bayesian-nns-using-pyro-and-pytorch-b1c24e6ab8cd>
 32. PyTorch NN Integration (Deep Kernel Learning) - GPyTorch's documentation, accessed January 18, 2026, https://docs.gpytorch.ai/en/v1.12/examples/06_PyTorch_NN_Integration_DKL/
 33. TYXE: PYRO-BASED BAYESIAN NEURAL NETS FOR PYTORCH - MLSys Proceedings, accessed January 18, 2026, https://proceedings.mlsys.org/paper_files/paper/2022/file/14bc46029b7ac590f56a203e0a3ef586-Paper.pdf
 34. Oriseven/DeepCRF_TIFS - GitHub, accessed January 18, 2026, https://github.com/Oriseven/DeepCRF_TIFS

35. Search | HyperAI, accessed January 18, 2026,
<https://beta.hyper.ai/search?q=Donggyu%20Joo>
36. bitzhangcy/Deep-Learning-Based-Anomaly-Detection - GitHub, accessed January 18, 2026,
<https://github.com/bitzhangcy/Deep-Learning-Based-Anomaly-Detection>